

Revision Notes & Predicted Exam Questions

Chapter 1 Data Representation | Chapter 2 Data Transmission | Chapter 3 Hardware
Chapter 8 Programming (Flowcharts & Pseudocode) | Chapter 10 Logic Gates

Part	Contents
A	Revision notes with worked examples (Ch 1, 2, 3)
B	Revision notes: flowcharts, pseudocode, trace tables, searching and sorting, logic gates
C	Predicted exam questions with full mark-scheme answers (28 questions)
D	Quick-fire self-test and last-minute checklist

How to use: read the notes, cover the green answer boxes in Part C, attempt each question under timed conditions (about 1 mark per minute), then mark yourself. Questions are *predicted* from the style and frequency of past Cambridge papers; they are not actual exam questions. Always check the current official syllabus for your exam year.

Part A. Chapter 1: Data Representation

1.1 Number systems

Computers use **binary** (base 2) because electronic circuits have two states (on/off = 1/0). **Hexadecimal** (base 16) is used by humans as a shorthand: one hex digit = 4 bits (a nibble), so it is shorter, easier to read and less error-prone than long binary strings.

Bit position (8-bit)	128	64	32	16	8	4	2	1
Example 10110111	1	0	1	1	0	1	1	1

Denary	0-9	10	11	12	13	14	15
Hex	0-9	A	B	C	D	E	F

Conversions (learn the method)

- **Binary to denary:** add the place values where there is a 1. $10110111 = 128+32+16+4+2+1 = 183$.
- **Denary to binary:** subtract place values from the left. $203 = 128+64+8+2+1 = 11001011$.
- **Binary to hex:** split into nibbles. $1011\ 0111 = B\ 7 = \mathbf{B7}$. **Hex to binary:** $3F = 0011\ 1111$.
- **Hex to denary:** $CB = (12 \times 16) + 11 = 203$. **Denary to hex:** $203 / 16 = 12\ r\ 11$, so CB.

Binary addition and overflow

Rules: $0+0=0$, $0+1=1$, $1+1=10$ (write 0 carry 1), $1+1+1=11$ (write 1 carry 1).

```
  01101011  (107)
+ 00110101  (53)
-----
 10100000  (160)   carries: 1 1 1 1 1 (on the lower bits)
```

Overflow happens when the result needs more bits than are available (e.g. an 8-bit register). $11001000\ (200) + 01100100\ (100) = 300$, which needs 9 bits, so the 9th bit is lost and the stored result is wrong ($00101100 = 44$).

Logical shifts

- Bits move left or right; vacated positions are filled with **0**; bits shifted out are lost.
- Left shift by n = multiply by 2^n ; right shift by n = divide by 2^n (integer part). Example: $00010110\ (22)$ left shift 2 = $01011000\ (88)$. Right shift 2 = $00000101\ (5)$; the two bits '10' are lost, so precision is lost.
- A left shift can cause overflow if a 1 is shifted out of the most significant end.

Two's complement (negative numbers)

In 8-bit two's complement the most significant bit has place value **-128**. To convert a positive number to its negative: **invert all bits, then add 1**. Example: $+45 = 00101101$, invert = 11010010 , add 1 = **11010011** (-45). Range of 8-bit two's complement: -128 to +127. Check: $-128+64+16+2+1 = -45$.

1.2 Representing text, sound and images

Data	How it is stored	Key points
Text	Each character has a unique binary code in a character set .	ASCII: 7-bit (128 chars) or 8-bit extended; 'A' = 65, 'a' = 97, '0' = 48. Unicode: 16/32 bit, supports all world languages and emojis, but uses more storage.
Sound	Analogue wave is sampled at regular intervals; each sample's amplitude is stored in binary.	Sample rate (Hz) = samples per second. Sample resolution (bit depth) = bits per sample. Higher values = better quality AND bigger file.
Images	A bitmap is a grid of pixels; each pixel stores a binary colour code.	Resolution = number of pixels (width x height). Colour depth = bits per pixel (1 bit = 2 colours, 8 bit = 256, 24 bit = 16.7 million). Metadata (width, height, colour depth) is stored in the header.

File size formulae

- **Image (bits)** = width x height x colour depth. Example: $800 \times 600 \times 24 = 11,520,000\ \text{bits} = 1,440,000\ \text{bytes} = \text{about } 1406\ \text{KiB}$.

- **Sound** (bits) = sample rate x resolution x length in seconds (x channels). Example: 44,100 x 16 x 60 = 42,336,000 bits = 5,292,000 bytes (about 5.05 MiB).
- Units: 8 bits = 1 byte; 1 nibble = 4 bits; 1 KiB = 1024 bytes; 1 MiB = 1024 KiB; 1 GiB = 1024 MiB; 1 TiB = 1024 GiB. Divide bits by 8 to get bytes.

1.3 Data storage and compression

Compression reduces file size so files **use less storage, upload/download faster and use less bandwidth**.

	Lossy	Lossless
Method	Permanently removes some data (e.g. reduces resolution, colour depth, sample rate, or removes sounds humans can't hear).	Data is re-encoded so the original can be rebuilt exactly (e.g. run-length encoding, RLE).
Quality / size	Smaller file, lower quality; cannot be restored.	Larger than lossy, no loss of quality.
Used for	Photos (JPEG), music (MP3), video (MP4).	Text, program files, medical images, PNG, ZIP.

RLE example: AAAABBBCCD becomes 4A3B2C1D (counts + value). Works best on data with many repeated values (e.g. a simple logo); can make data larger if there is little repetition.

Part A. Chapter 2: Data Transmission

2.1 Packets and packet switching

- Data is broken into small **packets**. Each packet has a **header** (destination IP address, sender's IP address, packet number/sequence, sometimes a time-to-live/hop count), a **payload** (the data) and a **trailer** (error-checking data / end marker).
- Packet switching**: each packet is sent independently and may take a **different route**, chosen by routers; packets may arrive out of order; at the destination they are **re-ordered** using the packet numbers and re-assembled.
- Advantages: better use of network, can avoid busy/broken routes, harder to intercept whole message.
Disadvantage: packets may be delayed or lost and need re-sending.

2.2 Methods of transmission

Term	Meaning	Notes
Serial	Bits sent one after another along a single wire.	Slower per cycle but reliable over long distances; no skew; cheaper cabling.
Parallel	Several bits sent simultaneously along multiple wires.	Faster over short distances; suffers skew (bits arrive at different times) over long distances; more costly.
Simplex	One direction only.	e.g. keyboard to computer, TV broadcast.
Half-duplex	Both directions but not at the same time.	e.g. walkie-talkie.
Full-duplex	Both directions at the same time.	e.g. telephone call, broadband.

USB (Universal Serial Bus): uses serial transmission. *Advantages*: universal standard, plug-and-play, backwards compatible, can only be inserted one way (newer connectors), supplies power, fairly fast, reliable. *Disadvantages*: limited cable length, slower than some other connections for large data, older versions have lower speeds.

2.3 Error detection

Errors occur during transmission through interference, data loss or data gain. Methods detect errors; the data is then re-sent.

Method	How it works	Limitations
Parity check	An extra parity bit makes the number of 1s even (even parity) or odd (odd parity). Receiver recounts the 1s. Even parity for 1011001 (four 1s): parity bit = 0. Odd parity: parity bit = 1.	Cannot detect two bits changing (or swapped) as the count stays the same; cannot locate the error. <i>Parity block</i> checks rows and columns to locate it.
Checksum	A value is calculated from the data block using an algorithm and sent with it. Receiver recalculates; if the two differ, an error occurred.	Some errors can leave the checksum unchanged.
Check digit	A digit calculated from the other digits (e.g. ISBN, barcode) and appended. Used for data entry errors (e.g. mistyped/transposed digits).	Not for transmission; some errors undetected.
Echo check	Receiver sends the data back to the sender, who compares it with the original.	If they differ the error could have been in either direction; slow (double transmission).
ARQ (Automatic Repeat reQuest)	Receiver sends a positive acknowledgement if the data is error-free, or a negative one if not. If no acknowledgement arrives within a timeout , the sender automatically re-sends. Repeats until acknowledged or a set number of attempts.	Needs a way to detect errors first (e.g. parity/checksum).

2.4 Encryption

- Encryption** scrambles **plaintext** into **ciphertext** using an algorithm and a **key**, so intercepted data cannot be understood. It does not stop interception; it stops the data being *read*.

- **Symmetric:** the **same (secret) key** encrypts and decrypts. Fast, but the key must be shared securely - if intercepted, the data is compromised.
- **Asymmetric:** a **public key** (shared openly) encrypts; a matching **private key** (kept secret) decrypts. Safer for key exchange, but slower. Used in HTTPS/TLS to set up a secure connection.
- Longer keys are harder to crack by brute force.

Part A. Chapter 3: Hardware

3.1 The CPU and computer architecture

- **CPU:** processes instructions. Contains: **ALU** (arithmetic and logic operations), **Control Unit** (CU, fetches/decodes/executes and coordinates), **registers** (tiny, very fast storage) and **buses**.
- **Von Neumann architecture:** instructions and data are stored together in the same main memory (RAM) as binary, in the same format; instructions are fetched one at a time (sequentially).
- **Registers:** PC (Program Counter - address of next instruction), MAR (Memory Address Register - address being read/written), MDR (Memory Data Register - data/instruction just fetched or about to be written), CIR (Current Instruction Register - instruction being decoded), ACC (Accumulator - holds results from the ALU).
- **Buses:** address bus (one-way, carries addresses), data bus (two-way, carries data/instructions), control bus (two-way, carries control signals e.g. read/write).

Fetch-decode-execute (FDE) cycle

Stage	What happens
1. Fetch	Address in PC is copied to MAR . The instruction at that address is sent along the data bus from RAM to MDR , then copied to CIR . PC is incremented to point to the next instruction.
2. Decode	The CU decodes the instruction in the CIR (opcode + operand) to work out what must be done.
3. Execute	The instruction is carried out (e.g. ALU does a calculation, data moved, a jump changes the PC). Results may be stored in the ACC or RAM. Then the cycle repeats.

Factors affecting CPU performance

- **Clock speed:** cycles per second (GHz). Higher = more instructions per second (but more heat).
- **Number of cores:** each core can run instructions at the same time, so multitasking and parallel programs are faster (a program must be written to use multiple cores).
- **Cache size:** small, very fast memory close to the CPU holding frequently used data/instructions, avoiding slower RAM access. Larger cache = fewer RAM fetches.

Embedded system: a computer system built into a larger device to do one dedicated task (e.g. washing machine controller, car engine management, microwave, traffic lights, digital watch). General-purpose computers (PCs) do many tasks.

Input/output devices: input - keyboard, mouse, touch screen, microphone, scanner, camera and **sensors** (temperature, light, pressure, moisture, infra-red, motion, pH, etc.); output - monitor, speakers, printers (inkjet, laser, 3D), actuators (motors), LED/LCD displays.

3.2 Data storage

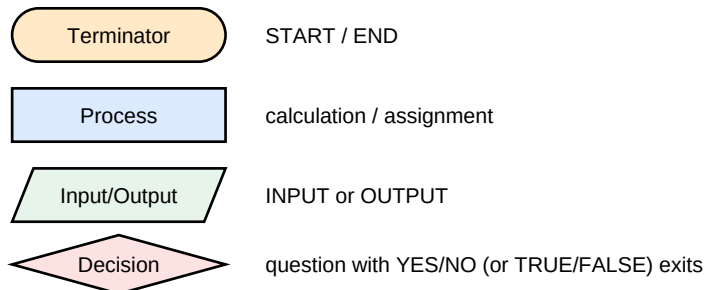
Type	Details
Primary: RAM	Volatile (contents lost when power off); read/write; holds currently running programs and data; larger RAM = more programs open smoothly.
Primary: ROM	Non-volatile; read-only; holds the bootstrap/start-up instructions (BIOS/firmware) .
Secondary - magnetic (HDD)	Spinning platters with a read/write head; cheap per GB, large capacity; moving parts so slower, noisier, uses more power, can be damaged by shock.
Secondary - solid state (SSD, flash, USB, SD)	Uses NAND flash (transistors/electrical charge); no moving parts so faster, silent, durable, low power; more expensive per GB; limited write cycles.
Secondary - optical (CD/DVD/Blu-ray)	Laser reads pits and lands on the disc surface; cheap and portable, but low capacity and slow; can scratch.
Virtual memory	When RAM is full, part of the secondary storage is used as if it were RAM. Pages of data are swapped between RAM and storage. Allows more programs to run, but is much slower; too much swapping causes thrashing .
Cloud storage	Data stored on remote servers accessed via the internet. + Access anywhere, automatic backup, scalable, no local hardware cost. - Needs internet, ongoing subscription cost, security/privacy concerns, depends on the provider.

3.3 Network hardware

- **NIC (Network Interface Card):** allows a device to connect to a network; gives the device a **MAC address** set by the manufacturer.
- **MAC address:** Media Access Control; unique, 48 bits, written as six pairs of hex digits (e.g. 00:1A:2B:3C:4D:5E); first half identifies the manufacturer, second half the device. Normally permanent (static), identifies the *device* on a local network.
- **IP address:** identifies a device's location on a network/the internet. **IPv4:** 32 bits, four numbers 0-255 (e.g. 192.168.0.1). **IPv6:** 128 bits, 8 groups of 4 hex digits, created because IPv4 addresses are running out. **Static** (fixed) or **dynamic** (assigned by the network each time you connect). Can be **public** or **private**.
- **Router:** connects networks together and forwards packets between them, choosing the best route; assigns local IP addresses and connects a home network to the internet (ISP).

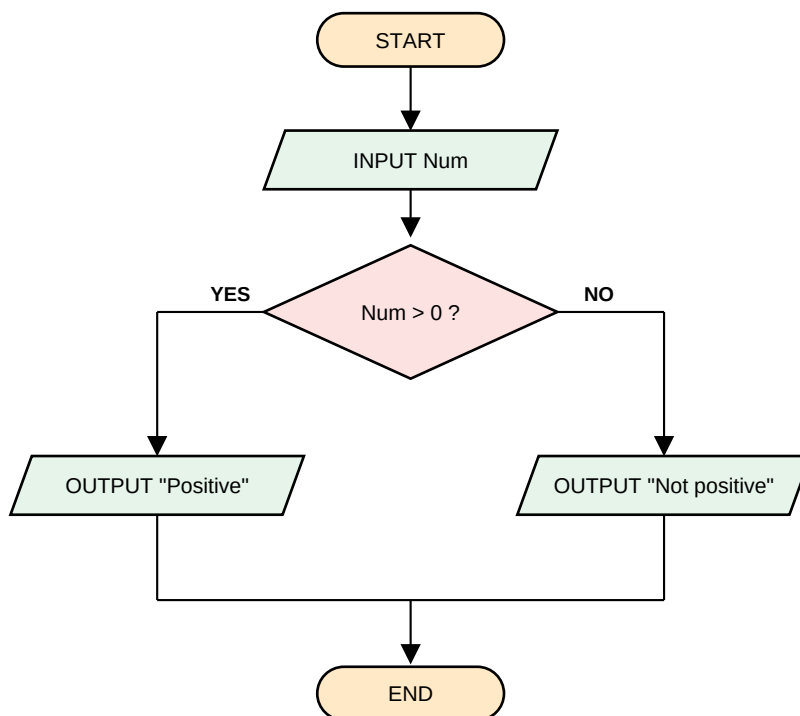
Part B. Chapter 8: Flowcharts, Pseudocode and Programming

8.1 Flowchart symbols



Rules: one START and one END; arrows show the flow (top to bottom/left to right); a decision has exactly **two labelled exits** (YES/NO); loops are shown by an arrow going back up to an earlier symbol.

Example flowchart: is a number positive?



8.2 Pseudocode reference (Cambridge style)

```
DECLARE Name : STRING                // data types: INTEGER, REAL, STRING, CHAR, BOOLEAN
CONSTANT Pi <- 3.142
Name <- "Sam"                        // assignment uses <-
INPUT Age                            // input
OUTPUT "Age is ", Age                // output

IF Age >= 18 THEN                     // selection
    OUTPUT "Adult"
ELSE
    OUTPUT "Minor"
ENDIF

CASE OF Choice                        // multi-way selection
```

```

1 : OUTPUT "One"
2 : OUTPUT "Two"
OTHERWISE : OUTPUT "Other"
ENDCASE

```

```

FOR i <- 1 TO 10                // count-controlled loop
  OUTPUT i
NEXT i

```

```

WHILE Total < 100 DO            // pre-condition loop (may run 0 times)
  Total <- Total + 5
ENDWHILE

```

```

REPEAT                          // post-condition loop (runs at least once)
  INPUT Pin
UNTIL Pin = 1234

```

- **Operators:** + - * / ^ (power), **DIV** (integer division, 17 DIV 5 = 3), **MOD** (remainder, 17 MOD 5 = 2); comparison = < > <= >=; Boolean AND, OR, NOT. **MOD** is used to test even/odd (Num MOD 2 = 0).
- **String functions:** LENGTH(s), LEFT(s, n), RIGHT(s, n), MID(s, start, n), UCASE(s), LCASE(s). **Maths:** ROUND(x, places), RANDOM().
- **Arrays:** DECLARE Marks : ARRAY[1:5] OF INTEGER, access with Marks[3]. 2D: ARRAY[1:3, 1:4], access Grid[2,3]. Loop through with FOR.
- **Totalling:** Total <- Total + Num. **Counting:** Count <- Count + 1.
- **Procedures and functions:** a procedure performs a task; a **function returns a value** (RETURN). Parameters pass values in. They make code reusable, shorter and easier to test.

```

FUNCTION Area(L : INTEGER, W : INTEGER) RETURNS INTEGER
  RETURN L * W
ENDFUNCTION

```

```

PROCEDURE ShowMenu()
  OUTPUT "1. Play"
  OUTPUT "2. Quit"
ENDPROCEDURE

```

```

OPEN "scores.txt" FOR WRITE      // file handling (modes: READ, WRITE, APPEND)
WRITEFILE "scores.txt", Score
CLOSEFILE "scores.txt"
OPEN "scores.txt" FOR READ
WHILE NOT EOF("scores.txt") DO
  READFILE "scores.txt", Line
  OUTPUT Line
ENDWHILE
CLOSEFILE "scores.txt"

```

Examiners accept any clearly structured pseudocode or a high-level language you have studied (Python, VB, Java); keep indentation and meaningful variable names.

8.3 Trace tables, validation and verification

- **Trace table:** a table with one column per variable (and output); write a new row every time a value changes, to follow an algorithm by hand and find logic errors.
- **Validation** checks data is *reasonable* (range, length, type, presence, format, check digit) - it does **not** prove the data is correct. **Verification** checks data was entered/copied *correctly* (double entry, visual check, proofreading, screen/print check).
- **Test data types:** **normal** (valid, expected), **abnormal/erroneous** (invalid, should be rejected), **extreme** (largest/smallest valid), **boundary** (on the edge of valid and invalid, e.g. 0, 1, 100, 101 for a range 1-100).
- **Error types:** syntax (breaks language rules), logic (wrong result, e.g. wrong operator), runtime (crash during running, e.g. divide by zero).

8.4 Searching and sorting algorithms

Linear search - checks each item in turn until found or the end is reached

```

Found <- FALSE

```

```

i <- 1
WHILE i <= 10 AND Found = FALSE DO
  IF Data[i] = Target THEN
    Found <- TRUE
  ELSE
    i <- i + 1
  ENDIF
ENDWHILE
IF Found THEN OUTPUT "Found at ", i ELSE OUTPUT "Not found" ENDIF

```

Works on unsorted or sorted lists; slow for large lists (worst case checks every item).

Bubble sort - repeatedly compares neighbouring items and swaps if out of order

```

FOR Pass <- 1 TO 4
  FOR i <- 1 TO 5 - Pass
    IF Data[i] > Data[i + 1] THEN
      Temp <- Data[i]
      Data[i] <- Data[i + 1]
      Data[i + 1] <- Temp
    ENDIF
  NEXT i
NEXT Pass

```

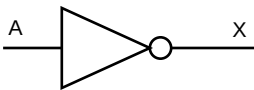
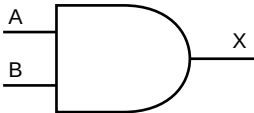
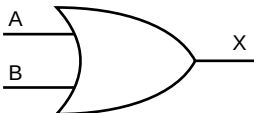
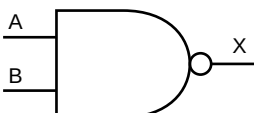
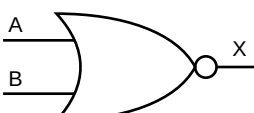
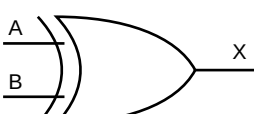
Pass 1 on 5, 2, 9, 1, 6: (5,2) swap -> 2,5,9,1,6; (5,9) no swap; (9,1) swap -> 2,5,1,9,6; (9,6) swap -> 2,5,1,6,9. The largest value 'bubbles' to the end each pass. An efficient version stops when a full pass makes no swaps (use a flag).

Computational thinking

Abstraction (removing unnecessary detail), **decomposition** (breaking a problem into smaller parts) and **pattern recognition** (spotting similarities to reuse solutions).

Part B. Logic Gates and Boolean Logic

Inputs and outputs are 1 (TRUE/ON) or 0 (FALSE/OFF). A truth table lists every input combination: n inputs give 2^n rows (2 inputs = 4 rows, 3 inputs = 8 rows). List inputs in binary counting order (000, 001, 010...).

Symbol	Gate / expression	Truth table															
	NOT X = NOT A <i>Inverts the input.</i>	<table><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	X	0	1	1	0									
A	X																
0	1																
1	0																
	AND X = A AND B <i>1 only if BOTH inputs are 1.</i>	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	0	1	0	1	0	0	1	1	1
A	B	X															
0	0	0															
0	1	0															
1	0	0															
1	1	1															
	OR X = A OR B <i>1 if AT LEAST ONE input is 1.</i>	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	1
A	B	X															
0	0	0															
0	1	1															
1	0	1															
1	1	1															
	NAND X = NOT (A AND B) <i>Opposite of AND: 0 only if both are 1.</i>	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	1	0	1	1	1	0	1	1	1	0
A	B	X															
0	0	1															
0	1	1															
1	0	1															
1	1	0															
	NOR X = NOT (A OR B) <i>Opposite of OR: 1 only if both are 0.</i>	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	1	0	1	0	1	0	0	1	1	0
A	B	X															
0	0	1															
0	1	0															
1	0	0															
1	1	0															
	XOR X = A XOR B <i>1 if the inputs are DIFFERENT.</i>	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	0
A	B	X															
0	0	0															
0	1	1															
1	0	1															
1	1	0															

Building truth tables for circuits (method)

- Write the expression from the circuit working left to right (e.g. $X = (A \text{ AND } B) \text{ OR } (\text{NOT } C)$).
- Add a column for each intermediate gate output, fill the table row by row, and finish with the final output column.
- Check the number of rows (2^n) and that you used a systematic order of inputs.

Example: $X = (A \text{ AND } B) \text{ OR } (\text{NOT } C)$

A	B	C	A AND B	NOT C	X
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Writing expressions from a description: 'an alarm sounds if the system is armed AND (the door OR the window sensor is triggered)' gives $X = A \text{ AND } (B \text{ OR } C)$. 'only if both switches are off' is a NOR; 'if the two inputs differ' is an XOR.

Part C. Predicted Exam Questions with Answers

Marks in brackets. Answers show the points an examiner looks for - each distinct, correct point earns a mark. Use the **command word**: *State* = short answer; *Describe* = say what happens; *Explain* = give reasons ('because...'); *Give* = list examples.

Chapter 1: Data Representation

Q1. (a) Convert the binary number 10110111 to denary. (b) Convert it to hexadecimal. (c) State one reason why hexadecimal is used. [3 marks]

Answer / mark scheme

(a) $128+32+16+4+2+1 = 183$ [1]

(b) **B7** [1]

(c) Shorter / easier for humans to read, remember and copy; fewer errors than binary; each hex digit = 4 bits so easy to convert [1]

Q2. (a) Add the binary numbers 01101011 and 00110101. (b) Explain what is meant by overflow, and state when it would occur if 11001000 and 01100100 were added in an 8-bit register. [4 marks]

Answer / mark scheme

(a) **10100000** [1 for the correct answer, 1 for showing the carries / working]

(b) The result of a calculation is too large to be stored in the available number of bits [1]. $200 + 100 = 300$ needs 9 bits so overflow occurs and the 9th (carry) bit is lost [1].

Q3. Show the 8-bit two's complement representation of -45. [2 marks]

Answer / mark scheme

+45 = 00101101 [1]. Invert and add 1: **11010011** [1]

Q4. The binary number 00010110 is shifted two places to the left using a logical shift. (a) Write down the result. (b) State the effect on the value. (c) Describe one problem that can occur with a left shift. [3 marks]

Answer / mark scheme

(a) **01011000** [1]

(b) The value is multiplied by 4 (22 becomes 88) [1]

(c) Overflow / a 1 bit may be shifted out of the register so the value becomes wrong [1]

Q5. A bitmap image is 800 pixels wide and 600 pixels high with a colour depth of 24 bits. (a) Calculate the file size in bytes. (b) Explain the effect on file size and quality of reducing the colour depth. (c) Name the type of compression that permanently removes data. [5 marks]

Answer / mark scheme

(a) $800 \times 600 \times 24 = 11,520,000$ bits [1]; $/ 8 =$ **1,440,000 bytes** (about 1406 KiB) [1]

(b) Fewer bits per pixel so the file size decreases [1]; fewer colours available so image quality is reduced / colours less accurate [1]

(c) **Lossy** compression [1]

Q6. Describe how sound is converted to a digital form, and explain the effect of increasing the sample rate. [4 marks]

Answer / mark scheme

The analogue sound wave is sampled / its amplitude measured at regular time intervals [1]; each measured value is converted to a binary number / stored [1]. Sample rate = number of samples taken per second. Increasing it gives a more accurate / better quality representation of the original wave [1] but a larger file size [1].

Q7. (a) Describe the difference between lossy and lossless compression. (b) Use run-length encoding to compress AAAABBBCCD. (c) Give one situation where lossless compression must be used. [5 marks]

Answer / mark scheme

(a) Lossy permanently removes data so the original cannot be restored; smaller file [1]. Lossless re-encodes data so the original can be rebuilt exactly; no loss in quality [1]

(b) **4A3B2C1D** [2: 1 for pairs of count+character, 1 for fully correct]

(c) Text documents / program code / medical images / legal documents - any data where every bit must be preserved [1]

Chapter 2: Data Transmission

Q8. Describe how a file is sent across the internet using packet switching. [5 marks]

Answer / mark scheme

The file is split into packets [1]. Each packet has a header containing the destination address / packet number / sender address [1]. Each packet may take a different route [1] decided by routers [1]. Packets may arrive out of order and are re-assembled at the destination using packet numbers [1]. (Lost/corrupted packets are re-requested.)

Q9. A byte 1011001 is sent with even parity. (a) State the parity bit. (b) The receiver gets 11011001 and finds an error - explain how. (c) Describe a limitation of a parity check. [4 marks]

Answer / mark scheme

(a) There are four 1s (even) so parity bit = **0** [1]

(b) The received byte has five 1s (odd) but even parity was agreed, so an error occurred [1]

(c) If two bits change the number of 1s stays even so the error is not detected / cannot say which bit is wrong [1]. Parity has to be agreed between sender and receiver [1 - optional].

Q10. Explain how an automatic repeat request (ARQ) is used to detect and correct errors in transmission. [4 marks]

Answer / mark scheme

Receiver checks the data for errors (e.g. parity/checksum) [1]. If no error, it sends a positive acknowledgement to the sender [1]. If an error is found it sends a negative acknowledgement (or none is sent) [1]. The sender waits for a timeout; if no positive acknowledgement arrives it re-sends the packet [1], repeating until it is received correctly or a limit is reached.

Q11. Explain the difference between symmetric and asymmetric encryption. [4 marks]

Answer / mark scheme

Symmetric uses the same key to encrypt and decrypt [1]; the key must be shared, so it could be intercepted [1]. Asymmetric uses a public key to encrypt and a different, private key to decrypt [1]; the private key is never shared, so it is more secure (but slower) [1].

Q12. Describe two differences between serial and parallel data transmission and state why USB uses serial. [4 marks]

Answer / mark scheme

Serial sends one bit at a time over one wire; parallel sends several bits at once over multiple wires [1]. Parallel is faster over short distances [1] but suffers skew over longer distances / serial is more reliable over distance [1]. USB uses serial because it is reliable, needs fewer wires/cheaper cables and avoids skew [1].

Chapter 3: Hardware

Q13. Describe the fetch-decode-execute cycle, naming the registers used. [5 marks]

Answer / mark scheme

Address in the PC is copied to the MAR [1]. The instruction at that address is fetched from RAM to the MDR [1] and copied to the CIR [1]. The PC is incremented [1]. The CU decodes the instruction and the ALU/CPU executes it [1].

Q14. State three factors that affect the performance of a CPU and explain one of them. [4 marks]

Answer / mark scheme

Clock speed, number of cores, cache size [1 each, max 3]. Explanation, e.g. more cores allow more instructions to be processed at the same time [1].

Q15. (a) State what is meant by an embedded system and give two examples. (b) Explain one difference between the Von Neumann model and an embedded system. [4 marks]

Answer / mark scheme

(a) A computer system built into a device to perform one dedicated function [1]; e.g. washing machine, microwave, traffic light controller, car engine management [1 for two]

(b) Von Neumann: general-purpose with data and instructions stored together in RAM and many programs can be run [1]; embedded system usually has a fixed program stored in ROM/firmware and does a single task [1].

Q16. Compare RAM and ROM. [4 marks]

Answer / mark scheme

RAM: volatile, read and write, stores currently used programs/data [2]. ROM: non-volatile, read-only, stores bootstrap / start-up firmware [2].

Q17. A laptop manufacturer is deciding between an SSD and an HDD. Give two advantages of an SSD and one advantage of an HDD, and name the technology used by an SSD. [4 marks]

Answer / mark scheme

SSD advantages (any two): faster read/write, no moving parts so more durable, silent, lower power consumption, lighter [2].

HDD advantage: cheaper per GB / larger capacity [1]. SSD uses NAND / flash memory (electrical charge in transistors) [1].

Q18. Explain what is meant by virtual memory and why it is needed. [4 marks]

Answer / mark scheme

Part of the secondary storage (hard drive/SSD) used as an extension of RAM [1]. When RAM is full, pages of data not currently needed are moved to virtual memory [1] and swapped back when needed [1]. Allows more/larger programs to be run than RAM could hold [1]. (It is slower than RAM.)

Q19. Describe the difference between a MAC address and an IP address. [4 marks]

Answer / mark scheme

MAC: assigned by the manufacturer to the NIC, permanent, unique 48-bit hex address identifying the device [2]. IP: assigned by the network/ISP, can change (dynamic), identifies the device's location on a network and is used for routing packets (IPv4 32-bit / IPv6 128-bit) [2].

Chapter 8: Flowcharts, Pseudocode and Programming

Q20. An algorithm inputs 10 student marks and outputs the average. Write pseudocode for this algorithm. [5 marks]

Answer / mark scheme

Initialise total to 0 [1]; loop 10 times [1]; input a mark and add to total inside the loop [1]; calculate average = total / 10 after the loop [1]; output the average [1].

```
Total <- 0
FOR Count <- 1 TO 10
    INPUT Mark
    Total <- Total + Mark
NEXT Count
Average <- Total / 10
OUTPUT Average
```

Q21. Complete a trace table for this algorithm using the inputs 4, 7, 10, 3, 6 and state the output. [6 marks]

```
Total <- 0
Count <- 0
FOR i <- 1 TO 5
    INPUT Num
    IF Num MOD 2 = 0 THEN
        Total <- Total + Num
        Count <- Count + 1
    ENDIF
NEXT i
OUTPUT Total, Count
```

Answer / mark scheme

See table: Total and Count change only when Num is even. **Output: 20, 3.** [1 for each correct row/column up to 5, 1 for the output]

i	Num	Total	Count
1	4	4	1
2	7	4	1
3	10	14	2
4	3	14	2
5	6	20	3

Output: 20 3 (the algorithm totals and counts the even numbers)

Q22. A program asks a user to enter a password. The password must be at least 8 characters. Write pseudocode that keeps asking until a valid password is entered. [5 marks]

Answer / mark scheme

Loop structure (REPEAT-UNTIL or WHILE) [1]; input inside the loop [1]; use of LENGTH [1]; correct condition (≥ 8 / < 8) [1]; error message output [1].

```
REPEAT
    OUTPUT "Enter a password"
    INPUT Password
    IF LENGTH(Password) < 8 THEN
        OUTPUT "Too short - try again"
    ENDIF
UNTIL LENGTH(Password) >= 8
OUTPUT "Password accepted"
```

Q23. An array Data[1:5] holds the values 5, 2, 9, 1, 6. (a) State the contents of the array after the first pass of a bubble sort. (b) Explain why bubble sort may need several passes. (c) Write pseudocode for a linear search of the array for a value entered by the user. [6 marks]

Answer / mark scheme

(a) **2, 5, 1, 6, 9** [2]

(b) Each pass only moves the largest remaining item to the end / compares neighbours only, so items may need to move several places; sorting finishes when a pass has no swaps [1]

(c) Input target [1]; loop through the array [1]; compare each element with the target [1]; output found/not found [1] (see Part B for full code).

Q24. Write pseudocode to store three names, entered by the user, in a text file called names.txt, then read them back and output them. [5 marks]

Answer / mark scheme

Open file for write [1]; loop 3 times with INPUT and WRITEFILE [1]; close the file [1]; open for read and loop reading with READFILE and OUTPUT [1]; close [1]. (Any clear syntax is accepted.)

```

OPEN "names.txt" FOR WRITE
FOR i <- 1 TO 3
    INPUT Name
    WRITEFILE "names.txt", Name
NEXT i
CLOSEFILE "names.txt"
OPEN "names.txt" FOR READ
WHILE NOT EOF("names.txt") DO
    READFILE "names.txt", Name
    OUTPUT Name
ENDWHILE
CLOSEFILE "names.txt"

```

Q25. (a) Explain the difference between validation and verification. (b) A program accepts a mark from 0 to 100. Give one example each of normal, abnormal and boundary test data. [6 marks]

Answer / mark scheme

(a) Validation checks that data is sensible / meets rules (range, type, length) but cannot prove it is correct [1]. Verification checks data has been entered or copied correctly (double entry, visual check) [1]

(b) Normal e.g. 57 [1]; abnormal e.g. -5 / 101 / 'abc' [1]; boundary e.g. 0, 100 (valid) and -1, 101 (invalid) [1]. Extreme data: 0 and 100 [1 optional].

Q26. Draw a flowchart (or describe the symbols you would use) for an algorithm that inputs a number and outputs 'Positive' if it is greater than 0, otherwise 'Not positive'. [3 marks]

Answer / mark scheme

START/END terminators [1]; INPUT in parallelogram, decision diamond 'Num > 0?' with YES/NO exits [1]; correct OUTPUT in parallelograms on each branch with arrows joining to END [1]. See the example flowchart in Part B.

Logic Gates

Q27. Complete the truth table for $X = A \text{ AND } (B \text{ OR } C)$. [3 marks]

A	B	C	X
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Answer / mark scheme

8 rows with A, B, C in order 000 to 111 [1]. X = 1 only for 101, 110 and 111 [2: all correct = 2, 1 error = 1].

A B C X	X = 1 only for rows 101, 110, 111
0 0 0 0	(A must be 1 AND at least one of B, C must be 1)
0 0 1 0	
0 1 0 0	
0 1 1 0	
1 0 0 0	
1 0 1 1	
1 1 0 1	
1 1 1 1	

Q28. A safety system has three inputs: A (door closed), B (guard in place) and C (emergency stop pressed). The machine runs ($X = 1$) only when A and B are both 1 and C is 0. (a) Write the logic expression. (b) Name the gate that gives 1 only when both inputs are 0. (c) Give the output of a NAND gate if both inputs are 1. [5 marks]

Answer / mark scheme

(a) $X = (A \text{ AND } B) \text{ AND } (\text{NOT } C)$ [3: 1 AND of A,B; 1 NOT C; 1 combining AND]

(b) **NOR** [1]

(c) **0** [1]

Part D. Quick-fire Self-test and Checklist

Question	Answer
Binary 11111111 in denary?	255
What is 1 nibble?	4 bits (one hex digit)
Hex for denary 255?	FF
Largest denary number in 8 bits (unsigned)?	255 ($2^8 - 1$)
Range of 8-bit two's complement?	-128 to +127
ASCII code for 'A'? 'a'?	65; 97
Bits needed for 100 colours?	7 ($2^7 = 128$)
Image size formula?	width x height x colour depth (bits)
Sound size formula?	sample rate x resolution x seconds
Which compression is reversible?	Lossless
What does a packet header contain?	Destination/source IP, packet number
Which error check re-sends data until acknowledged?	ARQ (with timeout)
Public key does what? Private key?	Encrypts; decrypts
Which transmission suffers from skew?	Parallel
Register holding the address of the next instruction?	PC (Program Counter)
Is RAM volatile?	Yes; ROM is non-volatile
Bits in IPv4 / IPv6 / MAC?	32 / 128 / 48
17 DIV 5? 17 MOD 5?	3; 2
Loop that always runs at least once?	REPEAT ... UNTIL
Flowchart decision symbol?	Diamond with two exits
Rows in a 4-input truth table?	16 (2^4)
NAND of 1 and 1?	0
XOR of 1 and 1?	0
NOR of 0 and 0?	1

Last-minute exam checklist

- Show **all working** in conversions, binary addition and file size calculations - method marks are common.
- For 'Explain' questions use **because / so / therefore**; give as many distinct points as there are marks.
- Do not write brand names; do not say 'faster' without saying what is faster.
- In pseudocode: use <- for assignment, indent loops and selections, close every IF, FOR, WHILE with ENDIF, NEXT, ENDWHILE.
- Trace tables: one column per variable, new row each time a value changes, record outputs in an output column.
- Truth tables: list inputs in counting order and use intermediate columns for each gate.
- Read the question for the **number of marks**: roughly one minute per mark; leave time to check.
- Practise past papers (Paper 1 and Paper 2) and use the mark schemes to learn the wording examiners reward.